Contromisure

Description

Contromisure: una prima classificazione

Le contromisure si dividono in tre principali categorie: **proattive**, **dinamiche** e **reattive**. Ognuna di queste ha il suo scopo e viene adottata in fasi diverse in base alla situazione di rischio. Un'altra classificazione più dettagliata, derivante dalla logica militare, include le seguenti fasi: **Prevenire, Resistere, Scoprire, Ripristinare, Reagire**.

1. Contromisure Proattive

Le contromisure **proattive** sono orientate a prevenire attacchi prima che possano verificarsi. Questo approccio si concentra sulla protezione dei sistemi e sull'identificazione delle vulnerabilità prima che vengano sfruttate.

Esempi di contromisure proattive includono:

- **Introduzione di controlli di sicurezza**: Modifiche strutturali per rafforzare il sistema e prevenirne l'accesso non autorizzato.
- **Rimozione di vulnerabilità**: Mitigazione delle vulnerabilità attraverso patching e aggiornamenti regolari.
- **Honeypots**: Sistemi finti creati per ingannare gli attaccanti e raccogliere informazioni sugli strumenti e le tecniche utilizzate.
- **Deception**: Tecniche di inganno per confondere l'attaccante e deviarlo dalle risorse reali.
- **C&C (Command & Control) Infrastructure**: Prevenzione dell'accesso a sistemi che potrebbero fungere da server di comando per attacchi remoti.
- **Ridondanza e Eterogeneità**: Utilizzo di sistemi eterogenei e ridondanti per ridurre i rischi di compromissione. La ridondanza fredda e calda sono applicazioni chiave in questo contesto.

2. Contromisure Dinamiche (Incident Response)

Le **contromisure dinamiche** sono legate alla **risposta agli incidenti**. Una volta che un attacco è stato identificato, le azioni intraprese per contenerlo, mitigarne l'impatto e fermarlo sono di natura dinamica.

In questa fase, le azioni includono:

- **Monitoraggio continuo** e analisi per identificare comportamenti sospetti.
- **Contenimento dell'intrusione**: Interruzione delle comunicazioni tra gli attaccanti e il sistema compromesso.
- **Isolamento di sistemi compromessi** per prevenire la propagazione dell'attacco.
- **Investigazione** per raccogliere dati sugli strumenti e le tecniche usate dall'attaccante.
- **Comunicazione interna ed esterna** per gestire la crisi in modo coordinato.

3. Contromisure Reattive

Le contromisure **reattive** intervengono dopo che un attacco ha avuto successo e si è verificata una compromissione. Queste risposte tendono a focalizzarsi sul **ripristino** del sistema al suo stato originale, minimizzando i danni.

Esempi di contromisure reattive:

- **Ripristino dei dati**: Recupero di dati da backup.
- **Ripristino della configurazione di sistema**: Ristabilire la configurazione sicura dei sistemi.
- **Ripristino dell'accesso**: Rimuovere l'accesso non autorizzato e risolvere le vulnerabilità sfruttate.
- **Analisi post-intrusione**: Rivedere il processo per imparare dalla compromissione e rafforzare i controlli.

Robustezza e Resilienza

La **robustezza** e la **resilienza** sono concetti fondamentali per le contromisure. La **robustezza** si riferisce alla capacità di un sistema di resistere a un attacco senza subire danni significativi, mentre la **resilienza** è la capacità di un sistema di **riprendersi rapidamente** dopo un attacco.

- **Ridondanza**:
  - **Fredda**: Moduli inattivi che vengono attivati quando quelli principali falliscono.
  - **Calda**: Moduli attivi che possono subentrare immediatamente in caso di guasto.
  - **Triple Modular Redundancy (TMR)**: Una forma di ridondanza in cui tre moduli eseguono la stessa operazione e poi "votano" per determinare l'output finale.
- **Eterogeneità**: L'uso di diversi tipi di componenti (ad esempio, sistemi operativi diversi) per ridurre il rischio che una vulnerabilità colpisca l'intero sistema.

Meccanismi Comuni alle Contromisure

Le contromisure sono implementate attraverso una serie di meccanismi condivisi, che includono:

- **Autenticazione e autorizzazione**: La gestione delle identità e dei diritti di accesso agli utenti.
- **Controllo sull'identità del soggetto**: Verifica dell'identità dell'utente (ad esempio, tramite password, biometria, OTP).
- **Gestione dei diritti**: Stabilire cosa un utente autenticato può fare all'interno del sistema.

Autenticazione e Autorizzazione

L'autenticazione e l'autorizzazione sono componenti fondamentali per la sicurezza informatica:

- **Autenticazione debole**: Utilizzo di password statiche, che sono vulnerabili agli attacchi di forza bruta e di sniffing.
- **Autenticazione forte**: Utilizzo di tecniche crittografiche per garantire la sicurezza dell'autenticazione, come il metodo challenge-response o l'uso di dispositivi fisici (ad esempio, token hardware).

Kerberos

Un esempio di sistema di autenticazione sicura è **Kerberos**, che utilizza chiavi segrete condivise per autenticare client e server. Il sistema si basa su un **Key Distribution Center (KDC)**, che gestisce la distribuzione di chiavi segrete e l'autenticazione degli utenti attraverso **ticket**. Questo processo permette di garantire che solo utenti legittimi possano accedere alle risorse protette.

Problemi di Kerberos

Kerberos è un sistema di autenticazione robusto, ma presenta alcuni problemi e limitazioni che possono influire sulla sua applicabilità in determinati contesti. I principali problemi includono:

1. Modifica delle Applicazioni (Kerberizzazione)

- Ogni programma che deve usare Kerberos richiede modifiche, un processo che può essere complesso, soprattutto in base alla tipologia di programma.
- La "kerberizzazione" di un'applicazione implica aggiungere il supporto per Kerberos, il che può richiedere l'accesso al codice sorgente dell'applicazione. In molti casi, non è possibile modificare o accedere ai sorgenti delle applicazioni, il che limita l'adozione di Kerberos in ambienti con software proprietario o non modificabile.

2. Dipendenza dalla Master Key

- Tutte le password e le chiavi nel sistema sono cifrate con una **master key** segreta. Questo è un punto critico di vulnerabilità: se il server di autenticazione (KDC) viene compromesso, tutte le password salvate sono a rischio, e sarebbe necessario cambiare tutte le password.
- Un attacco o una vulnerabilità che comprometta la **master key** potrebbe compromettere l'intero sistema, poiché la sicurezza di tutte le credenziali dipende da questa chiave.

3. Single Point of Failure: Server di Autenticazione

- Se il server di autenticazione (KDC) dovesse cadere o diventare inaccessibile, l'intera rete che usa Kerberos diventa inutilizzabile. Ciò rappresenta un **single point of failure** critico: la disponibilità del sistema è fortemente dipendente dal corretto funzionamento di questo componente.

- Per mitigare questo problema, sono necessarie soluzioni di alta disponibilità per il KDC, ma ciò aumenta la complessità e i costi dell'implementazione.

4. Rilevazione di Intrusioni

- La **rilevazione di intrusioni** è difficile per l'utente, poiché Kerberos non fornisce meccanismi di monitoraggio in tempo reale o avvisi sugli attacchi. La gestione delle intrusioni è quindi demandata ad altri strumenti di sicurezza, come i sistemi di rilevamento delle intrusioni (IDS), ma non è integrata direttamente in Kerberos.

5. Problemi in Ambienti Time-Sharing

- In ambienti di **time-sharing** come i sistemi Unix, Kerberos memorizza i ticket nella directory **/tmp**, che è una directory pubblica a cui ogni utente può accedere. Questo significa che i ticket potrebbero essere rubati da un attaccante che ha accesso a quella directory, aumentando il rischio di compromissione.
- Sebbene si possano implementare misure per limitare l'accesso a /tmp, questa è una debolezza nota che potrebbe esporre a rischi aggiuntivi.

6. Vita Limitata dei Ticket

- I ticket di Kerberos hanno una **durata limitata** (di solito 8 ore), il che può rappresentare un problema per le applicazioni che richiedono tempi di esecuzione più lunghi. Questo può comportare la necessità di implementare politiche di **gestione e rinnovo dei ticket**.
- Se un'applicazione non può rinnovare il proprio ticket in tempo, l'accesso alle risorse potrebbe essere interrotto, causando disagi operativi. Le applicazioni devono essere progettate per gestire il rinnovo dei ticket, ma ciò aggiunge complessità al sistema.